

Data Analytics Tasks:

A. Basic Aggregation:

1. Find the total number of purchases per product per year and list down the top selling products every year

Query:

```
WITH product_rank AS(
SELECT
    YEAR(STR_TO_DATE(ph.purchase_date, '%m/%d/%Y %H:%i')) AS
    purchase_year, pd.product_id, COUNT(ph.purchase_date) AS
    purchase_count, pd.category,
    RANK() OVER (PARTITION BY
    YEAR(STR_TO_DATE(ph.purchase_date, '%m/%d/%Y %H:%i'))
    ORDER BY COUNT(ph.purchase_date) DESC) AS rank_per_year
FROM
    purchase_history_dataset ph
JOIN
    products_dataset pd ON ph.product_id = pd.product_id
GROUP BY
    pd.product_id, purchase_year, pd.category
)
SELECT
    purchase_year, product_id, category, purchase_count
FROM
    product_rank
WHERE
    rank_per_year = 1
ORDER BY
    purchase_year, product_id;
```

Output: Done in SQL & Output is in excel table

2. Calculate the average quantity purchased per product and list down the top 5 products with high avg quantity purchased

Query:

```
SELECT
    ph.product_id, pd.product_name, avg(ph.quantity) AS avg_quantity
FROM
    purchase_history_dataset ph
JOIN
    products_dataset pd ON ph.product_id = pd.product_id
GROUP BY
    ph.product_id, pd.product_name
```

```
ORDER BY
    Avg_Quantity DESC
LIMIT 5;
```

Output: Done in SQL & Output is in excel table

B. Join Operations:

1. Join purchase history with products dataset to get the product name for each purchase.

Query:

```
SELECT
    ph.purchase_id, ph.product_id, pd.product_name, ph.purchase_date,
    ph.quantity, ph.total_amount
FROM
    purchase_history_dataset ph
JOIN
    products_dataset pd ON ph.product_id = pd.product_id;
```

Output: Done in SQL & Output is in excel table

2. Join purchase history with customer profile dataset to include customer information for each purchase and list top 5 customers with high purchases

Query:

```
WITH TP AS(
SELECT
    cp.customer_id, COUNT(ph.purchase_date) AS total_purchase,
    DENSE_RANK() OVER (ORDER BY COUNT(ph.purchase_date)
    DESC) AS rank_tp
FROM
    purchase_history_dataset ph
JOIN
    customer_profile_dataset cp ON ph.customer_id = cp.customer_id
GROUP BY
    cp.customer_id
)
SELECT
    TP.customer_id, cp.first_name, cp.last_name, cp.email, cp.phone_number,
    TP.total_purchase
FROM
    customer_profile_dataset cp
JOIN
    TP ON TP.customer_id = cp.customer_id
WHERE
```

```

rank_tp<=5
ORDER BY
total_purchase DESC;

```

Output: Done in SQL & Output is in excel table

C. Window Functions:

1. Find the cumulative sum of purchases for each product category over year.

Query:

```

WITH category_n_year_sum AS(
SELECT
    pd.category, YEAR(STR_TO_DATE(ph.purchase_date, '%m/%d/%Y
%H:%i')) AS purchase_year,
    SUM(ph.total_amount) AS year_wise_total_amount
FROM
    purchase_history_dataset ph
JOIN
    products_dataset pd
ON
    ph.product_id = pd.product_id
GROUP BY
    pd.category, YEAR(STR_TO_DATE(ph.purchase_date, '%m/%d/%Y
%H:%i'))
ORDER BY
    pd.category, YEAR(STR_TO_DATE(ph.purchase_date, '%m/%d/%Y
%H:%i'))
)
SELECT
    *, SUM(year_wise_total_amount) OVER (PARTITION BY Category
ORDER BY purchase_year) AS cumulative_sum
FROM
    category_n_year_sum;

```

Output: Done in SQL & Output is in excel table

D. Rank and Dense Rank:

1. Rank customers based on their total expenditure.

Query:

```

WITH TPM AS(
SELECT
    cp.customer_id, SUM(ph.total_amount) AS total_purchase_amount
FROM

```

```

        purchase_history_dataset ph
JOIN
    customer_profile_dataset cp ON ph.customer_id = cp.customer_id
GROUP BY
    cp.customer_id
)
SELECT
    TPM.customer_id, cp.first_name, cp.last_name, total_purchase_amount,
    RANK() OVER (ORDER BY total_purchase_amount DESC) AS
    rank_on_TPM
FROM
    customer_profile_dataset cp
JOIN
    TPM ON TPM.customer_id = cp.customer_id
ORDER BY
    total_purchase_amount DESC;

```

Output: Done in SQL & Output is in excel table

2. Identify the top 10 customers by purchase frequency using `DENSE_RANK()`.

Query:

```

WITH CustomerPurchaseFrequency AS (
    SELECT
        customer_id,
        COUNT(purchase_id) AS purchase_frequency
    FROM
        purchase_history_dataset
    GROUP BY
        customer_id
),
RankedCustomers AS (
    SELECT
        customer_id,
        purchase_frequency,
        DENSE_RANK() OVER (ORDER BY purchase_frequency
        DESC) AS ranking
    FROM
        CustomerPurchaseFrequency
)
SELECT
    cp.customer_id,
    cp.first_name,
    cp.last_name,
    cp.email,
    cp.phone_number,

```

```

        rc.purchase_frequency,
        rc.ranking
FROM
    RankedCustomers AS rc
JOIN
    customer_profile_dataset AS cp
ON
    rc.customer_id = cp.customer_id
WHERE
    rc.ranking <= 10
ORDER BY
    rc.ranking, rc.purchase_frequency DESC;

```

Output: Done in SQL & Output is in excel table

E. Percentiles:

1. Calculate the 25th, 50th (median), and 75th percentiles of total purchase amounts for each customer segment.

Query:

```

WITH customer_total_purchase AS (
SELECT
    ph.customer_id,
    SUM(ph.total_amount) AS total_purchase_amount
FROM
    purchase_history_dataset ph
GROUP BY
    ph.customer_id
),
ranked_purchases AS (
SELECT
    total_purchase_amount,
    NTILE(4) OVER (ORDER BY total_purchase_amount) AS quartile
FROM
    customer_total_purchase
)
SELECT
    CASE
        WHEN quartile = 1 THEN '25th'
        WHEN quartile = 2 THEN '50th'
        WHEN quartile = 3 THEN '75th'
    END AS percentile,
    MAX(total_purchase_amount) AS value
FROM
    ranked_purchases

```

```
GROUP BY
    quartile
HAVING
    quartile IN (1, 2, 3);
```

Output: Done in SQL & Output is in excel table

F. Median Calculation:

1. Compute the median purchase amount per product category.

Query:

```
WITH OrderedData AS (
SELECT
    pd.category,
    ph.total_amount,
    ROW_NUMBER() OVER (PARTITION BY pd.category ORDER BY
    ph.total_amount) AS row_num,
    COUNT(*) OVER (PARTITION BY pd.category) AS total_rows
FROM
    purchase_history_dataset ph
JOIN
    products_dataset pd ON ph.product_id = pd.product_id
),
MedianData AS (
SELECT
    category,
    total_amount AS median_amount
FROM
    OrderedData
WHERE
    -- For odd number of rows, select the middle one
    row_num = (total_rows + 1) / 2
OR
    -- For even number of rows, average the two middle ones
    row_num IN (total_rows / 2, (total_rows / 2) + 1)
)
SELECT
    category,
    AVG(median_amount) AS median_purchase_amount
FROM
    MedianData
GROUP BY
    category;
```

Output: Done in SQL & Output is in excel table

G. Complex Aggregation:

1. Find the average, maximum, and minimum purchase value for each product type and customer age group.

Age groups: Under 20, 20-40, Above 40

Query:

```
WITH calculate_age AS(
SELECT
    cp.customer_id,
    STR_TO_DATE(cp.date_of_birth, '%m/%d/%Y %H:%i') AS
    date_of_birth,
    ROUND((DATEDIFF(CURDATE(), STR_TO_DATE(cp.date_of_birth,
'%m/%d/%Y')))/365) AS Age
FROM
    customer_profile_dataset cp
),
age_category AS(
SELECT
    customer_id,
    CASE
        WHEN Age < 20 THEN "Under 20"
        WHEN Age >=20 and Age<=40 THEN "20 to 40"
        WHEN Age > 40 THEN "Above 40"
    END AS age_group
FROM calculate_age
ORDER BY customer_id
)
SELECT
    pd.category, age_group,
    AVG(ph.total_amount) AS average_amount,
    MAX(ph.total_amount) AS max_amount,
    MIN(ph.total_amount) AS min_amount
FROM
    age_category ac
JOIN
    purchase_history_dataset ph
ON
    ac.customer_id = ph.customer_id
JOIN
    products_dataset pd
ON
    ph.product_id = pd.product_id
GROUP BY
    pd.category, age_group
```

```
ORDER BY
    pd.category;
```

Output: Done in SQL & Output is in excel table

H. Grouping:

1. Group purchases by day of the week and find the average number of purchases made on each day.

Query:

```
SELECT
    DAYNAME(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i')) AS
    day_of_week, AVG(total_amount) as avg_of_total_amount
FROM
    purchase_history_dataset
GROUP BY
    day_of_week
ORDER BY
    FIELD(day_of_week, 'Sunday', 'Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday');
```

Output: Done in SQL & Output is in excel table

2. Group customers by city and find the total number of purchases and total revenue generated.

Query:

```
SELECT
    cp.city,
    COUNT(ph.purchase_id) AS total_no_of_purchases,
    SUM(ph.total_amount) AS total_revenue_generated
FROM
    customer_profile_dataset cp
JOIN
    purchase_history_dataset ph
ON
    ph.customer_id = cp.customer_id
GROUP BY
    cp.city
ORDER BY
    total_revenue_generated DESC;
```

Output: Done in SQL & Output is in excel table

I. Case Statement:

1. Classify customers as “High-Spending,” “Medium-Spending,” or “Low-Spending” based on their total purchase amounts (using percentiles in multiples of 33).

Query:

```
WITH Total_Spending AS(
    SELECT
        ph.customer_id, cp.first_name, cp.last_name,
        sum(ph.total_amount) as total_purchase_amt
    FROM
        purchase_history_dataset ph
    JOIN
        customer_profile_dataset cp ON ph.customer_id = cp.customer_id
    GROUP BY
        ph.customer_id, cp.first_name, cp.last_name
),
spending_percentile AS(
    SELECT
        *, NTILE(3) OVER (ORDER BY total_purchase_amt) AS
        spending_category
    FROM
        Total_Spending
)
SELECT *,
CASE
    WHEN spending_category =1 THEN "Low-Spending"
    WHEN spending_category =2 THEN "Medium-Spending"
    WHEN spending_category =3 THEN "High-Spending"
END AS Customer_Segment
FROM
    spending_percentile
ORDER BY
    customer_id;
```

Output: Done in SQL & Output is in excel table

J. Join with Condition:

1. Join purchase history with customer profile dataset where the customer’s age is above 30, and display their purchase details.

Query:

```
WITH customer_age AS (
    SELECT
```

```

        customer_id, FLOOR(DATEDIFF(CURRENT_DATE,
        STR_TO_DATE(date_of_birth, '%m/%d/%Y %H:%i')) / 365.25) AS age
FROM
    customer_profile_dataset
)
SELECT
    ph.purchase_id, ph.customer_id, ca.age, ph.product_id, ph.purchase_date,
    ph.quantity, ph.total_amount
FROM
    purchase_history_dataset ph
JOIN
    customer_age ca ON ph.customer_id = ca.customer_id
WHERE
    ca.age > 30;

```

Output: Done in SQL & Output is in excel table

K. Top N Analysis:

1. Find the top 5 products contributing to the highest revenue.

Query:

```

SELECT
    pd.product_id,
    pd.product_name,
    SUM(ph.total_amount) AS revenue_generated
FROM
    products_dataset pd
JOIN
    purchase_history_dataset ph
ON
    pd.product_id = ph.product_id
GROUP BY
    pd.product_id, pd.product_name
ORDER BY
    revenue_generated DESC
LIMIT 5;

```

Output: Done in SQL & Output is in excel table

2. Identify the top 3 cities with the most number of unique customers.

Query:

```

SELECT
    city, COUNT(DISTINCT customer_id) AS no_of_customers
FROM

```

```

        customer_profile_dataset
GROUP BY
    city
ORDER BY
    no_of_customers DESC
LIMIT 3;

```

Output: Done in SQL & Output is in excel table

L. Window Functions for Trend Analysis:

1. Create a 7-day moving average of total purchases per product and which customer had high 7 day MA

Query:

```

SELECT
    product_id,
    STR_TO_DATE(purchase_date,'%m/%d/%Y %H:%i') AS purchase_date,
    SUM(total_amount) AS total_amount,
    AVG(SUM(total_amount)) OVER (
        PARTITION BY product_id
        ORDER BY STR_TO_DATE(purchase_date,'%m/%d/%Y
        %H:%i')
        ROWS BETWEEN CURRENT ROW AND 6 FOLLOWING
    )AS seven_day_moving_avg
FROM
    purchase_history_dataset
GROUP BY
    product_id,
    purchase_date
ORDER BY
    product_id, purchase_date;

```

Output: Done in SQL & Output is in excel table

L. Sub question: To find which customer had high 7 day MA:

Query:

```

WITH Moving_average_customerwise AS(
    SELECT
        customer_id,
        STR_TO_DATE(purchase_date,'%m/%d/%Y %H:%i') AS
        purchase_date,
        SUM(total_amount) AS total_amount,
        AVG(SUM(total_amount)) OVER (
            PARTITION BY customer_id

```

```

        ORDER BY STR_TO_DATE(purchase_date,'%m/%d/%Y
        %H:%i')
        ROWS BETWEEN CURRENT ROW AND 6
        FOLLOWING
    )AS seven_day_moving_avg
FROM
    purchase_history_dataset
GROUP BY
    customer_id,
    purchase_date
ORDER BY
    customer_id, purchase_date
),
Ranked_Customers AS (
    SELECT
        customer_id,
        purchase_date,
        total_amount,
        seven_day_moving_avg,
        RANK() OVER (ORDER BY seven_day_moving_avg DESC) AS
        ranking
    FROM
        Moving_average_customerwise
)
SELECT
    rc.customer_id,
    cp.first_name,
    cp.last_name,
    cp.email,
    cp.phone_number,
    rc.purchase_date,
    rc.total_amount,
    rc.seven_day_moving_avg
FROM
    Ranked_Customers rc
JOIN
    customer_profile_dataset cp ON rc.customer_id = cp.customer_id
WHERE
    rc.ranking = 1
ORDER BY
    rc.customer_id,
    rc.purchase_date;

```

Output: Done in SQL & Output is in excel table

M. Nested Queries:

1. Find customers who have never purchased products from the top 5 most popular categories.

Query:

According to the datasets provided we have only 4 categories, but in the question it is mentioned as “top 5 most popular categories”. To overcome this I have written a query to return data according to top 2 most popular categories.

```
SELECT
    cp.customer_id, cp.first_name, cp.last_name
FROM
    customer_profile_dataset cp
WHERE
    cp.customer_id NOT IN
    (SELECT DISTINCT ph.customer_id
    FROM
        purchase_history_dataset ph
    JOIN
        products_dataset pd ON ph.product_id = pd.product_id
    WHERE
        pd.category IN
        (SELECT
            category
        FROM (
            SELECT
                pd.category, COUNT(ph.purchase_id) AS
                no_of_purchases
            FROM
                purchase_history_dataset ph
            JOIN
                products_dataset pd
            ON
                ph.product_id = pd.product_id
            GROUP BY
                pd.category
            ORDER BY
                no_of_purchases DESC
            LIMIT 2
        ) AS TopCategories
    )
);
```

Output: Done in SQL & Output is in excel table

2. Identify products purchased only once in the entire dataset.

Query:

According to our dataset no product is purchased once. The minimum number of purchase is 8 for product_id 216. So the below query will return 0 rows.

```
SELECT
    DISTINCT ph.product_id, pd.product_name
FROM
    purchase_history_dataset ph
JOIN
    products_dataset pd
ON
    ph.product_id = pd.product_id
WHERE
    ph.product_id IN
        (SELECT
            ph.product_id
        FROM
            purchase_history_dataset ph
        GROUP BY
            ph.product_id
        HAVING
            COUNT(ph.purchase_id)=1
        );
```

Output: Done in SQL & Output is in excel table

N. Date Analysis:

1. Identify the month with the highest total sales volume.

Query:

```
SELECT
    MONTHNAME(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i'))
    AS Month_Name,
    SUM(total_amount) AS total_purchase_amount
FROM
    purchase_history_dataset
GROUP BY
    Month_Name
ORDER BY
    total_purchase_amount DESC
LIMIT 1;
```

Output: Done in SQL & Output is in excel table

2. Calculate the year-over-year growth of total sales.

Query:

```
WITH Yearwise_total_amount AS(
SELECT
    YEAR(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i')) AS years,
    SUM(total_amount) AS total_purchase_amount
FROM
    purchase_history_dataset
GROUP BY
    years
ORDER BY
    years
)
SELECT
    years,
    total_purchase_amount,
    CEIL(
        ((total_purchase_amount - LAG(total_purchase_amount) OVER
        (ORDER BY years)) /
        LAG(total_purchase_amount) OVER (ORDER BY years)) * 100
    ) AS yoy_growth_percentage
FROM
    Yearwise_total_amount;
```

Output: Done in SQL & Output is in excel table

O. Join with Aggregation:

1. Join all three datasets to find the total revenue per product per customer.

Query:

```
SELECT
    cp.customer_id, pd.category, SUM(ph.total_amount) AS total_revenue
FROM
    customer_profile_dataset cp
JOIN
    purchase_history_dataset ph
ON
    cp.customer_id = ph.customer_id
JOIN
    products_dataset pd
ON
    ph.product_id = pd.product_id
GROUP BY
    cp.customer_id, pd.category
```

```
ORDER BY
    cp.customer_id;
```

Output: Done in SQL & Output is in excel table

R. Subqueries:

1. Identify customers whose total expenditure is above the average expenditure of all customers.

Query:

```
SELECT
    customer_id,
    first_name,
    last_name,
    total_expenditure
FROM (
    SELECT
        cp.customer_id, cp.first_name, cp.last_name,
        SUM(ph.total_amount) AS total_expenditure
    FROM
        purchase_history_dataset ph
    JOIN
        customer_profile_dataset cp
    ON
        ph.customer_id = cp.customer_id
    GROUP BY
        cp.customer_id, cp.first_name, cp.last_name
    ORDER BY
        cp.customer_id
) AS customer_expenditures
WHERE
    total_expenditure > (SELECT AVG(total_expenditure)
    FROM (
        SELECT
            customer_id,
            SUM(total_amount) AS total_expenditure
        FROM
            purchase_history_dataset
        GROUP BY
            customer_id
        ) AS avg_expenditures
    );
```

Output: Done in SQL & Output is in excel table

2. Find products with a purchase amount higher than the average purchase amount across all products.

Query:

```
SELECT
    product_id,
    product_name,
    total_expenditure
FROM(
    SELECT
        pd.product_id, pd.product_name, SUM(ph.total_amount) AS
        total_expenditure
    FROM
        purchase_history_dataset ph
    JOIN
        products_dataset pd
    ON
        ph.product_id = pd.product_id
    GROUP BY
        pd.product_id, pd.product_name
    ORDER BY
        pd.product_id
) AS product_expenditures
WHERE
    total_expenditure > (SELECT AVG(total_expenditure)
    FROM (
        SELECT
            product_id,
            SUM(total_amount) AS total_expenditure
        FROM
            purchase_history_dataset
        GROUP BY
            product_id
        ) AS avg_expenditures
    );
```

Output: Done in SQL & Output is in excel table

S. Correlated Sub queries:

1. Find the purchase dates where a customer made a purchase larger than their average purchase amount.

Query:

```
SELECT
    ph.customer_id,
```

```

        ph.purchase_date,
        ph.total_amount AS purchase_amount,
        avg_per_customer.avg_purchase_amount
FROM
    purchase_history_dataset AS ph
JOIN (
    SELECT
        customer_id,
        AVG(total_amount) AS avg_purchase_amount
    FROM
        purchase_history_dataset
    GROUP BY
        customer_id
    ) AS avg_per_customer
ON
    ph.customer_id = avg_per_customer.customer_id
WHERE
    ph.total_amount > avg_per_customer.avg_purchase_amount
ORDER BY
    ph.customer_id, ph.purchase_date;

```

Output: Done in SQL & Output is in excel table

2. List all products that were purchased more frequently than the average frequency of products in their category.

Query:

```

WITH ProductFrequency AS (
    SELECT
        ph.product_id,
        p.category,
        COUNT(ph.purchase_id) AS purchase_frequency
    FROM
        purchase_history_dataset AS ph
    JOIN
        products_dataset AS p
    ON
        ph.product_id = p.product_id
    GROUP BY
        ph.product_id, p.category
),
CategoryAverageFrequency AS (
    SELECT
        category,
        AVG(purchase_frequency) AS avg_category_frequency
    FROM

```

```

        ProductFrequency
    GROUP BY
        category
    )
    SELECT
        pf.product_id,
        pd.product_name,
        pf.category,
        pf.purchase_frequency,
        caf.avg_category_frequency
    FROM
        ProductFrequency AS pf
    JOIN
        CategoryAverageFrequency AS caf
    ON
        pf.category = caf.category
    JOIN
        products_dataset AS pd
    ON
        pf.product_id = pd.product_id
    WHERE
        pf.purchase_frequency > caf.avg_category_frequency
    ORDER BY
        pf.category, pf.purchase_frequency DESC;

```

Output: Done in SQL & Output is in excel table

T. Date Functions:

1. Extract the day, month, and year from the purchase date, and group total purchases by month across all years

Query:

```

    SELECT
        YEAR(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i')) AS years,
        MONTHNAME(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i'))
        AS months,
        DAY(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i')) AS days,
        DAYNAME(STR_TO_DATE(purchase_date, '%m/%d/%Y %H:%i')) AS
        name_of_the_day,
        SUM(total_amount) AS total_purchases
    FROM
        purchase_history_dataset
    GROUP BY
        years, months, days, name_of_the_day
    ORDER BY

```

years, FIELD(months, 'January', 'February', 'March', 'April', 'May', 'June',
'July', 'August', 'September', 'October', 'November', 'December'), days;

Output: Done in SQL & Output is in excel table